
LAB 4 : BEST-FIT CONTIGUOUS MEMORY ALLOCATION

OCC : 1

GROUP : 8

INTRODUCTION TO BEST-FIT CONTIGUOUS MEMORY ALLOCATION

Contiguous memory allocation is a management technique where memory is divided into fixed-sized partitions, each containing exactly one process of sufficient size.

Formula

If a process starts at memory address a and has a size of s units, it occupies addresses $a, a+1, a+s-1, \dots$

Primary use

simulate the way the operating system manages main memory by deciding which free blocks to allocate to processes in a multiple-partition environment.

KEY CHARACTERISTICS

Fixed-Sized Partitions

system divides memory into several partitions, each capable of housing exactly one process at a time.

Allocation Tracking

The operating system maintains an internal table that identifies which specific parts of memory are currently available and which are occupied.

Size Optimization

The Best-fit strategy specifically identifies and selects the memory block that is closest in size to the process's requirements.

Partition Recycling

Once a process finishes execution and terminates, its partition is automatically released and becomes available for the next process in the input queue.

WORKING PRINCIPLE

01

Search and Identification:

When a process arrives and needs memory, the operating system searches the available memory sections to identify all free blocks that are large enough to accommodate the request

02

Selection Logic:

If more than one free block of sufficient size exists, the Best-fit strategy evaluates the options and chooses the specific block that is closest in size to the request.

03

Loading and Release:

Once the best-fitting block is selected, the process is loaded into that partition, and upon its termination, the system updates the memory table to mark the partition as free.

CODE IMPLEMENTATION

```
// Best-Fit algorithm
for (int j = 0; j < numJobs; j++) {
    int bestIdx = -1;
    int bestDiff = Integer.MAX_VALUE;

    // using sortedIdx
    for (int i : sortedIdx) {
        if (!occupied[i] && blockSize[i] >= jobSize[j]) {
            int diff = blockSize[i] - jobSize[j];
            if (diff < bestDiff) {
                bestDiff = diff;
                bestIdx = i;
            }
        }
    }

    if (bestIdx == -1) {
        jobError[j] = "Job " + (j + 1) + " (" + jobSize[j] + " KB) - no sufficient block available.";
    } else {
        occupied[bestIdx] = true;
        allocatedJob[bestIdx] = j + 1;
        allocatedJobSize[bestIdx] = jobSize[j];
        jobBlock[j] = bestIdx;
        jobError[j] = null;
    }
}
```


PROGRAM SAMPLE OUTPUT

run:

Enter total number of memory blocks: 3

Block 1 size (KB): 100

Block 2 size (KB): 20

Block 3 size (KB): 40

=== MEMORY LIST ===

Memory Location	Block Size (KB)	Status	Job No.	Job Size (KB)

2	20 KB	FREE	-	-
3	40 KB	FREE	-	-
1	100 KB	FREE	-	-

Enter total number of jobs: 4

Job 1 size (KB): 30

Job 2 size (KB): 90

Job 3 size (KB): 21

Job 4 size (KB): 10

PROGRAM SAMPLE OUTPUT

```
=== ALLOCATION ERRORS ===
  [!] Job 3 (21 KB) - no sufficient block available.

=== UPDATED MEMORY LIST ===
Memory Location   Block Size (KB)   Status   Job No.   Job Size (KB)
-----
2                20 KB          OCCUPIED   Job 4      10 KB
3                40 KB          OCCUPIED   Job 1      30 KB
1               100 KB          OCCUPIED   Job 2      90 KB

=== ALLOCATION SUMMARY ===
Total Memory:                160 KB
Total Memory Blocks:         3 blocks
Total Jobs Submitted:        4 jobs
Jobs Successfully Allocated:  3 jobs
Jobs Failed (No Fit):        1 jobs
Memory Used by Jobs:         130 KB
Internal Fragmentation:      30 KB
Free Memory Remaining:       0 KB
Free Blocks Remaining:       0 blocks
BUILD SUCCESSFUL (total time: 34 seconds)
```

THANKYOU
